

P?S/8 Version 8Z Change Log.

Copyright © 1988-2017 »CLA Systems
CLASystems@gmail.com
Last edit: 09-May-2017 cjl

All changes to P?S/8 Version 8Z are logged in this file.

23-Feb-1988 Initial system creation.

This is the date of the initial release of P?S/8 Version 8Z.

This does not include updates to various components created somewhat later. The relevant files are currently under review; however, the only known issues relate to the simultaneous use of the Logical Console Overlay configured for the VT8E and FOCAL customized with the VT8E-oriented VT8PCH binary patch file. This issue has been tabled pending a more general release of the system.

13-Nov-2014 Version 8Z for MDC8 diskette (FLP) image recovered.

This image was recovered from a working diskette for P?S/8 Version 8Z configured for the MDC8 5.25" diskette drives (FLP).

Most importantly, the .DUMP files that are used to generate system components beyond the initial starter system were recovered from this image after successfully placing it into the SIMH environment. (Note: A series of utilities was required to create the SIMH version of this image. The files were accessed and converted using existing bootable images of P?S/8 Version 8U.)

All known variant system-generation .DUMP files were recovered including the common file used to extend the starter system (created from binary files of the relevant source files) to a (nearly) complete system. (Note: Several additional system components are generally added to full releases to include support for specific hardware such as media formatting or copying programs.)

The .DUMP file used to complete the FLP system is essentially the same as the similar file used for TC01/TC08/TU55/TU56 systems with the exception of the device length (the capacity of the FLP system diskettes exceeds that of standard or premium DECTapes).

26-Nov-2014 P?S/8 Version 8Z "A" source files recovered.

All known source files for P?S/8 that are part of the "A" collection (meant to be placed on an RKAX: drive or equivalent) were recovered from OS/8 format MDC8 5.25" diskettes. Image files were transferred to SIMH and all files were checked for consistent contents and file validity. These files have also been transferred to MS-DOS/windows equivalent directories with full file names (OS/8 file naming limitations have been removed).

P?S/8 Version 8Z Change Log (continued).

13-Dec-2014 P?S/8 Version 8Z "B" source files recovered.

All known source files for P?S/8 that are part of the "B" collection (meant to be placed on an RKBx: drive or equivalent) were recovered from OS/8 format MDC8 5.25" diskettes. Image files were transferred to SIMH and all files were checked for consistent contents and file validity. These files have also been transferred to MS-DOS/windows equivalent directories with full file names (OS/8 file naming limitations have been removed).

At some point later, the VT8E-related source files mentioned above were recovered along with many other files unrelated to P?S/8 (apparently as part of a general backup of an OS/8 development drive). These files are not part of the general release of P?S/8 per se; they are needed by users who configure the Logical Console Overlay and/or P?S/8 FOCAL for use with the VT8E DMA terminal device; binary files produced from these files are passed to the respective program for customization purposes. Future releases of P?S/8 may include these updated source files; at this time, it is not known if this work was finalized.

As of this writing, nearly all P?S/8 source files have been recovered with notable exceptions related to the MDC8 winchester (hard drive) system, the two variant LINC-8 system configurations supported, and all files related to the TD8E both with and without the MR8E-C ROM.

The MDC8 winchester source files can be derived from other source files already recovered, notably the P?S/8 MDC8 FLP system and non-system device handlers, and the OS/8 MDC8 system and non-system device handlers for the FLP and winchester systems.

Bootable LINC-8 systems are available on LINCtapes. The missing source files can be recovered by binary disassembly; additionally, certain related utilities are available in source form.

All P?S/8 TD8E source files are presently unavailable. Assuming the files are never recovered, programming for this device will require new work, in part based on the author's recollections.

Note: A partial recovery of TD8E support has been initiate using a recovered P?S/8 TD8E with MR8E-C ROM DECTape image. It is not clear if this is the final version of this configuration. The author acknowledges this particular system variant was produced under expedient circumstances with known defects. For the present, it is presumed this will be the best available recovered source code fragment; disassembly has produced a working copy that documents the known defective behavior.

P?S/8 Version 8Z Change Log (continued).

Additionally, a set of specific workarounds for this aspect of this system have been proposed but not finalized pending overall source code cleanup; it is not certain the corrected source code will be compatible with P?S/8 Version 8Z, as most new work in ongoing projects is targeted to a next-generation Version 9A system which has slightly different internal features. Considering the changes that will be required to produce Version 9A compatibility, it is deemed of little value to produce an interim release for Version 8Z. Additionally, some of the recovery work will affect a TD8E with extended memory variant system, as certain components are either partially or fully compatible across both configurations.

It is known that the missing files were never part of the "A" and "B" file collections due to lack of sufficient storage space at the various development sites used at the time. Backup copies of these files may show up as part of an ongoing recovery process.

All present efforts will be focused on releasing other P?S/8 configurations that are achievable using the various source files, most notably the TC01/TC08/TU55/TU56 DECTape system, the TC12 LINCTape system, the RX01 system and the RK8E/RK05 system. Several other sets of handler source code exists for somewhat obscure third-party devices such as the DSD-240-8/Western Dynex fixed/removable disk cartridge system.

The most appropriate system to implement first is a modern release of the TC01/TC08/TU55/TU56 system; surviving hardware is available in several locations and there is excellent support for this device within the SIMH environment. Many internal aspects of P?S/8 development must be dealt with before any semblance of a workable system can be created.

15-May-2015 Hacking up a bootable system for the TC01/TC08 DECTape.

Using P?S/8 Version 8U for TC01/TC08/TU55/TU56 available on the SIMH simulator, the system handler for the TC01/TC08/TU55/TU56 was kludged to a copy of the image of the Version 8Z FLP system. Predictably, many functions do not work properly; with some manual patching of the directory, a viable temporary system was created.

Most importantly, access to certain newer system programs is now possible to aid in a proper reconstruction. (Note: P?S/8 Version 8U is quite incompatible with Version 8Z; the only functional level of compatibility is the keyboard monitor commands to access the TFS directory on the newer system.)

P?S/8 Version 8Z Change Log (continued).

15-Jun-2015 Producing a proper bootable startup system for the TC01/TC08 DECTape.

Various OS/8 BATCH files are available within the "A" and "B" file collections to create the binary files needed to create a P?S/8 release. By studying the BATCH files, it was determined what source files are needed to create what is referred to as the P?S/8 "starter" system.

The starter system is a bootable operating system with limited capability configured for the target device . Creation of the starter system depends on the ability to run the target system device handler for the purpose of creating a clean image of itself (and other system components) from a working copy moved to the intended operating memory locations (07600-07777). An internal system generation routine is used, essentially a series of calls to write out system component sections loaded into most memory locations of a 12K machine; the system generation routine is aware of the loading address of all components involved in the system generation (which has nothing to do with their ultimate execution address when called into memory for their normal operating function).

Note: Certain system device configurations require an extended memory section for normal operation. In certain cases, the complication of creating a functioning system device handler for the intended purpose may be cumbersome. Future releases will avoid this problem by supplying an associated non-system device handler for the identical configuration. In most instances, the non-system device handler will occupy less memory than the routines needed to force temporary viability of the system device handler. Additionally, in certain specific cases, the source code can be simplified to allow internal documentation that is more appropriate to the resultant coding.

The starter system is obtained by loading the following binary files in the proper order:

- 1) Z12K.BN This is a memory-clearing file that loads 0000 into every memory location in the range of 00000-07577, 10000-17577 and 20000-27577. This pre-clears the memory that will be occupied by the system components that are loaded next.
- 2) PQSMON.BN This is the primary code of the keyboard monitor, P?S/8 BATCH, FIND, CHANGE and various support elements of P?S/8 that must be written to designated blocks. The system generation routine is also part of this file.

P?S/8 Version 8Z Change Log (continued).

- 3) PQSDTA.BN This is the image of the system device handler for TC01/TC08/TU55/TU56. Since this device also supports a *slurp* loader, the image of the *slurp* loader is included. (Note: For systems that do not support a *slurp* loader, the system generation routine writes out a deliberately non-functional dummy program stub to make attempts to load a *slurp* loader instead use the standard virtual loader.) Since this device supports a *slurp* loader, the system device handler reloader "file" image [associated with the BIN /I option switch] is also included.
- 4) BIN.BN This is the image of the BIN utility program which includes the START and GET commands created as aliases.
- 5) ODT.BN This is the image of the P?S/8 ODT debugging program which is a proper superset of the OS/8 equivalent.

With ODT available to the starter system, it is possible to make certain changes to the keyboard monitor and other components if unusual situations arise. [Note: The starter system is quite meager without the additional utilities to be added later.]

- 6) DUMP.BN This is the image of the P?S/8 DUMP utility program. Recent releases of DUMP support command files to automate block transfers and block patching. Once the starter system is running, .DUMP files can be passed to DUMP to create the rest of the system. [Note: The system generation process creates a safe subset of the TFS directory without supporting the proper capacity; the appropriate .DUMP file is used to initialize the TFS directory parameters to match the proper device size. Advanced users can override these values if appropriate; this is usually used in situations such as supporting premium-length DECTapes and LINCTapes.]\
[Note: The .DUMP files are generally made available on a logical device unit other than the system bootup unit which is created without any existent files. The files can be copied to the system bootup unit if required; at least one additional logical device unit is required for the binary data to be transferred as the starter system is updated.]

The starter system binary composite is started at

00200. The machine halts to allow any manual changes of target media; pressing `CONTINUE` starts the system generation process. After many write operations, the machine halts; pressing `CONTINUE` will boot the newly-created starter system.

25-Jun-2015 Problems completing the common system generation.

while not all components of the OS/8 BATCH command files have been tested to determine if a complete collection of required binary files can be obtained, a problem has already been identified:

All components of every released configuration of P?S/8 beyond the starter system must be created on an OS/8 non-system device such as DTA1:. Even if a specific program is not required for any particular system, error-free assembly of all components must be enforced.

The reason for this is that the OS/8 non-system device will create core-image copies of all of the P?S/8 components involved; the specific OS/8 records where these files are stored is crucial. The .DUMP files are extremely pragmatic; the most minor change will corrupt the process.

With two notable exceptions, all of the P?S/8 source files are compatible with PAL8 Version B0 which is the standard version released with OS/278 Version 2.0. To within a minor operational bug that never affects most users, this version will function correctly under earlier systems such as OS/8 Version 3D, considered the standard release for all PDP-8 systems. (The bug has to do with the TTY: handler and interaction with the `Control-Z` character; the problem is completely avoidable by observing proper timing or simply not using the TTY: handler.)

Two programs, MARK12 and TC12F, are adaptations of standard components of the DIAL-MS system for the PDP-12 only. The source code was taken from a specific release of DIAL-MS and is likely not the final release code. Apparently there are no operational differences between this code and the standard versions found on any newer DIAL systems per se; however, these programs have been modified to be more suitable for use with 128 words/block LINCtapes (which are used by P?S/8 and OS/8 on the PDP-12 and the LINC-8) with the ability to format longer LINCtapes assuming premium media is available. As such, these programs must be supported as written, if possible.

P?S/8 Version 8Z Change Log (continued).

(Note: Custom modifications exist in the P?S/8 version of the programs to overcome DIAL-specific issues which are in conflict with PDP-8 conventions. When these programs are in memory, the P?S/8 kernel is destroyed. As such, manual bootstrap procedures are required after using either program.)

When P?S/8 system generation was being performed in 1988, these programs were assembled with a specific user-written assembler known as PAL12.

PAL12 is based on a very early release of PAL8 that is quite defective. Despite major shortcomings, the LINC mode assembly was an acceptable implementation of the DIAL assembler LINC mode subsection. Very little PDP-8 code is used in these programs; as such, PAL12 was an acceptable assembler for this limited purpose. (Note: P?S/8 PAL is a proper superset of both PAL8 and PAL12; however, it is not an OS/8 program.)

Unfortunately, attempts to locate this particular release of PAL12 has not been successful:

Earlier releases of PAL12 were located that cannot assemble the programs due to lacking certain assembler features added later. Apparent file dates confirm these are precursors to the stable release required.

Unfortunately, the PAL12 project was turned over to other programmers who have a very poor understanding of exactly how a LINC mode assembler operates. While the features missing from the earlier releases are present, the ability to assemble most LINC instructions is hopelessly incompatible; the newer release is essentially worthless for practical assembly of most LINC programs.

A binary copy of OS/8 for the modified 8K LINC-8 has been located. This includes LINC-8 handlers loaded into a copy of OS/8 BUILD. This LINCtape may be the only surviving OS/8 LINCtape for the LINC-8 in the world. A paper directory indicates the proper PAL12 release is on this tape. Once a reasonable recovery process can be completed, the issue of producing the common system components [and system-specific addons] will be revisited.

Note: Some programming effort [using the SIMH PDP-8 simulator] has been performed modifying the MARK12 program in an attempt to make the source code compatible with the newer [albeit defective] release of PAL12. In theory, after a lot of effort, the source code for the two affected programs could produce proper binary files for use in the P?S/8 system generation process. However, the two affected programs are not present on the P?S/8 Version 8Z [FLP] system; existing binary copies are needed to confirm the source code changes produce the intended binary output. A P?S/8 Version 8R system for TC12 has also been recovered. If a copy of the core image of the programs on this LINCtape can be transferred to the SIMH environment, the source code

modification process can continue with confidence.

P?S/8 Version 8Z Change Log (continued).

27-Jun-2015 Hacking up a complete P?S/8 Version 8Z system for TC08.

It has been determined that the P?S/8 Version 8Z [FLP] system is essentially the same as the TC08 system with regard to the process of creating the common system generation and system-specific extensions. The only important difference is ensuring the TFS directory length is appropriate to the TC08 system. [Note: This was determined by inspection of the FLP-related system-specific extensions .DUMP file and the TC08-related system-specific extensions .DUMP file.]

P?S/8 BLKCPY was run by booting to the starter system with the hacked-up TC08 [but mostly FLP] system on logical device unit 6. This allowed the major sections that would be created by the .DUMP files to be copied en masse from the blocks on the hacked-up image on device unit 6 to the corresponding blocks on the booted device unit 0.]

[Note: While P?S/8 is fully compatible with all standard bootstraps on all standard devices, the TC01/TC08/TU55/TU56 Version 8Y and newer systems are also compatible with TCBOOT, an extended DECTape startup program that allows P?S/8 to boot to any DECTape drive unit in the range of 0 through 7. TCBOOT can also boot most PDP-8 operating systems for the same configuration as long as drive unit 0 is selected. While fully functional, this is not recommended since shorter bootstraps exist for drive 0 only. A copy of TCBOOT is available from Charles Lasner; this version requires manual drive unit selection via the front panel switches; an automatic version can be derived that will detect the first drive available in a manner analogous to the extended RXBOOT program for RX01/RX02/RX03/DSD-210.]

Certain structures were also copied using P?S/8 DUMP in manual command mode, such as copying the system programs directory; this allows the newly-added systems program to run directly on the newly-created TC08 system.

After confirming [by direct inspection with P?S/8 DUMP] that all directory structures are consistent with the TC08 system, the first P?S/8 Version 8Z system for TC01/TC08/TU55/TU56 in over twenty-five years has been produced!

10-Jul-2015 P?S/8 PAL bug fix.

Problem: PAL creates defective binary files only during an obscure scenario involving the simultaneous creation of binary files and listing output at the same time. If literals are in use and there are no current page literals on the last page of code and page zero literals exist to be dumped because there is a FIELD directive in the source code, the field change associated with the new field setting will be created incorrectly. The problem had apparently been present for many years prior to Version 8Z, but was never observed before due to the requirement of so many specific conditions.

P?S/8 Version 8Z Change Log (continued).

The problem was traced to an improper use of a temporary location that is unavailable in the specific circumstances described above.

Resolution: As a temporary fix, all references to the temporary [XP45CH] will be patched to instead use an available page zero location; Future releases will reorganize the code to prevent the problem.

Implementation: Using P?S/8 BLKODT, the problem is readily fixed:

.BLKODT

0:0266.001 /3315 3174

11+

0:0266.012 /1315 1174

S

.

01-Aug-2015 Date year base group update to 2008.

In P?S/8 Version 8Z, the date is partially determined by the present value of the implicit base year group. The base year group is calculated as the count of 8 year increments past 1960; when P?S/8 Version 8Z was released, the value used was 3 corresponding to a valid year range of 1984 through 1994. [Note: while the internal implementation of the date allows a theoretical range of somewhat over 11 years, the P?S/8 DATE utility only allows valid dates in the present 11 year range to avoid unexpected date expiration during the next year following.]

The base year values are implemented as internal locations within all programs that support the present date; the relative date within the 11 year range is stored in the system kernel as the contents of location 07610. The system kernel value is updated by the DATE command; the other programs generally support printing today's date by reading the current date value in memory and calculating the overall date using the locally-stored base year group value in the image of the program. As such, all programs require a binary patch to reflect the desired base year group value.

The replacement value to be applied is 6, which corresponds to a date range of 2008 through 2018. If desired, the base year group value can be updated in the future to 7 to correspond to a date range of 2016-2026. [Note: P?S/8 Version 9A will use an entirely different method of implementing the system date. The entire date will be available in the system kernel; the maintenance procedure described here will be obsolete.]

P?S/8 Version 8Z Change Log (continued).

Implementation: Using P?S/8 BLKODT, the base year group values are readily updated:

.BLKODT

0:0255.162	/0003	6	PAL
0:0363.000	/0003	6	MAP
0:0414.000	/0003	6	DATE
0:0440.000	/0003	6	PRINT
0:0613.140	/0003	6	SYSTAT
0:0627.021	/0003	6	MAP

S
.

After all patches are applied, the current date can be set with the DATE command. Operation of all affected programs should be tested for correct output.

05-Aug-2015 LE{ft} command message change.

In keeping with current practice of character counting, the definition of the basic unit of textual input in the P?S/8 keyboard monitor and integrated editor is being changed from characters to bytes.

Since the P?S/8 text file format uses six-bit ASCII codes with several translated characters, the proper reference term will be the six-bit byte; this is consistent with all available command documentation.

As such, the message associated with the count of available characters within the text file currently in the edit buffer will be changed from

CHARACTERS: xxxx xxxx is the free character count

to

BYTES: xxxx xxxx is the free byte count

Implementation: Using P?S/8 BLKODT, the message text characters are readily updated:

.BLKODT

0:0007.171	/7764	7771	message length
0:0007.172	/0310	0231	BY
0:0007.173	/0122	2405	TE
0:0007.174	/0103	2372	S:
0:0007.175	/2405	4000	<SP><EOL>

S
.

P?S/8 Version 8Z Change Log (continued).

Notes:

- 1) This patch frees up locations 01576-01577 within the keyboard monitor for a projected patch presently under review.
- 2) Using the P?S/8 OS8CON program, P?S/8 six-bit bytes can be converted from/to seven-bit ASCII text bytes as used in OS/8 and other systems [and eventually the P?S/8 SHELL overlay]. The present P?S/8 six-bit character set supports various languages including P?S/8 PAL and the P?S/8 implementation of FOCAL, 1969. Future releases are expected to support implementations of BASIC and FORTRAN.

08-Aug-2015 Keyboard monitor bug fix.

Problem: Keyboard monitor commands that support both an optional *:u* suffix and an optional *filename* [such as EN{ter}, FE{tch}, KI{ll} and WR{ite}] fail to properly support the *filename* when both options are used simultaneously. [Note: The problem also exists if the *filename:u* form is used; however, the second *:u* suffix is unrelated to the problem.]

In most instances, the optional *:u* suffix is rarely used. When the optional *filename* is not present, the default name from a prior command is used as the file name for the current command:

.FE FOOTBAR The explicit file name used sets the default name.

[Some editing operations are performed to update the file contents.]

.WR The updated file is written using the default name.

Alternatively, the command

.WR:1

writes the file to logical device unit 1 with the same file name.

Since there is no explicit file name in either WR{ite} command example, both commands function as expected.

However, if both options are used in the same command such as

.WR:1 FOOTBAR

the command will fail to complete properly and will likely lead to an error message. [Note: The contents of the edit buffer are not affected by the problem. A similar command with one of the options or no options will succeed.]

P?S/8 Version 8Z Change Log (continued).

The problem was traced to a failure to properly update all internal command parsing data; this can only happen when the simultaneous usage of both options is attempted.

Resolution: As a temporary fix, the proper call to the optional unit scan routine [NEWCAT] is patched in using available locations on the same code page. Future releases will reorganize the code to prevent the problem using less total words.

Implementation: Using P?S/8 BLKODT, the problem is readily fixed:

.BLKODT

0:0007.166 /5401 5376

0:0007.176 /2223 4134

0:0007.177 /7240 5401

S

.

Notes:

- 1) Implementation of this patch depends on the prior implementation of the 05-Aug-2015 LE{ft} command message change as described above.
- 2) As of this writing, this patch has not received rigorous testing. The only P?S/8 Version 8Z configuration presently released is for TC01/TC08/TU55/TU56; as such, the release file pqs8_tc08.tu56 [image file for use with the SIMH PDP-8 simulator] will also be available without this patch; the unpatched file will be known as pqs_tc08_05-Aug-2015.tu56 consistent with the previous patch level.

19-Aug-2015 P?S/8 PAL bug fix.

Problem: The ASMIFM, ASMIFN and ASMIFZ directives are non-functional.

Note: These directives are part of the dual mode assembly option and were added for compatibility with the DIAL-MS assembler.

Detailed explanation.

The total failure of the three directives mentioned above was traced to an incorrect offset past a symbolic reference within the source code due to gradual changes in the coding.

P?S/8 Version 8Z Change Log (continued).

This is actually a poor programming technique that is no longer used in P?S/8 development. If a specific location will be modified, it should have its own symbolic identification. In this case, the referenced address is part of a relocation list. [The dual mode assembly feature is one of the modular options; depending on the specific combination of modules retained, a dynamic update reflecting the relocation must be applied to all memory locations in the rest of the assembler coding that need to be updated accordingly.] In this specific case, the relocation was correct; however, it was applied to an incorrect memory location.

Resolution: As a temporary fix, the proper address will be patched to restore proper operation. The source code will be modified to include a proper symbolic reference [without a potentially "drifting" offset] to ensure the accuract of the coding. [At the source code level, the address requiring the relocation was erroneously coded as MASSIF+2 when in fact it should be MASSIF+4 due to changes in the source code over the years. A new reference to NASSIF without an offset will be applied, as this is presently the closest symbolic address.]

Implementation: Using P?S/8 BLKODT, the problem is readily fixed:

```
.BLKODT
```

```
0:0277.010 /2373 2375
S
.
```

Note: Conditional directives such as IFZERO, IFNZRO, IFDEF and IFNDEF are universally available in P?S/8 PAL regardless of modular options chosen [such as dual mode assembly].

The DIAL assembler supports the ASMIFM, ASMIFN, ASMIFZ and ASMSKP directives as an equivalent functionality to the IFZERO and IFNZRO directives; however, the usage is somewhat more cumbersome [in part due to the usage of one's complement arithmetic when assembling in LINC mode]. The standard directives can handle equivalent parameters to achieve the same results, generally using a simpler overall approach. [The IFDEF and IFNDEF directives have no functional equivalent in the DIAL assembler.]

Since the standard directives are fully compatible with all popular PAL assemblers [such as TOPS-10 PAL10, OS/8 PAL8 and P?S/8 PAL], it is recommended to avoid the DIAL assembler directives in all new work.

As such, support for the DIAL conditional directives will be available in P?S/8 PAL only as part of the dual mode assembly module.

Additional P?S/8 PAL issues:

No fix for the following problems will be made available for this release of P?S/8 PAL. A minimal workaround is discussed below.

P?S/8 Version 8Z Change Log (continued).

Certain directives are implemented in the dual mode assembly module that arguably should be available regardless of modular options set; the only way to use the directives is by enabling dual mode assembly with the /8 or /9 option switches.

The ENBITS and NOBITS directives are available for compatibility with TOPS-10 PAL10; however, there is no actual function performed (as the feature within PAL10 the directives manage is not implemented in any other PAL assembler). (The intended usage is to enable or disable the automatic bitmap dump after every FIELD directive usage. OS/8 PAL8 and P?S/8 PAL do not support this implementation; however, both systems support an independent bitmap facility to perform an equivalent function on the binary output of the assembly. As such, it is not likely that functional compatibility will be available in either system.)

Since these directives are not particularly related to dual assembly mode, the next source-level release of P?S/8 PAL will shift these directives into the main symbol table load. For the present, if these features are required, the restriction of requiring the /8 option switch be set will apply.

The DTORG directive is also erroneously part of the dual mode assembly module; however, DTORG should not be used as it will tend to corrupt the binary output. (There had been an intention to create an alternate special-purpose loading utility to punch binary paper-tapes of output containing DTORG statements, but as a practical matter, there no longer is any interest in the Typeset-8 product, the only system where this feature was ever required. However, future releases of P?S/8 will provide an alternate implementation that is also fully compatible with all binary output standards. At the point of release, the directive will be moved to the main symbol table load in the same manner as the NOBITS and ENBITS directives as discussed above. Future P?S/8 system generation utilities will require the revamped DTORG directive; it will be possible to convert the compatible binary output to the format as required by Typeset-8 punched on incompatible binary paper tapes.)

31-Dec-2015 Date year base group update to 2016.

In P?S/8 Version 8Z, the date is partially determined by the present value of the implicit base year group. The base year group is calculated as the count of 8 year increments past 1960; when P?S/8 Version 8Z was released, the value used was 3 corresponding to a valid year range of 1984 through 1994. [Note: While the internal implementation of the date allows a theoretical range of somewhat over 11 years, the P?S/8 DATE utility only allows valid dates in the present 11 year range to avoid unexpected date expiration during the next year following.]

The base year values are implemented as internal locations within all programs that support the present date; the relative date within the

11 year range is stored in the system kernel as the contents of location 07610. The system kernel value is updated by the DATE command; the other programs generally support printing today's date by reading the current date value in memory and calculating the overall date using the locally-stored base year group value in the image of the program. As such, all programs require a binary patch to reflect the desired base year group value.

The replacement value to be applied is 7, which corresponds to a date range of 2016 through 2026. If desired, the base year group value can be updated in the future to 10 to correspond to a date range of 2024-2034. [Note: P?S/8 Version 9A will use an entirely different method of implementing the system date. The entire date will be available in the system kernel; the maintenance procedure described here will be obsolete.]

Implementation: Using P?S/8 BLKODT, the base year group values are readily updated:

.BLKODT

0:0255.162 /0003 7	PAL
0:0363.000 /0003 7	MAP
0:0414.000 /0003 7	DATE
0:0440.000 /0003 7	PRINT
0:0613.140 /0003 7	SYSTAT
0:0627.021 /0003 7	MAP
S	
.	

After all patches are applied, the current date can be set with the DATE command. Operation of all affected programs should be tested for correct output.

04-Apr-2017 Reversal of Patch.

08-Aug-2015 Keyboard monitor bug fix.

Problem: Keyboard monitor commands that support both an optional *:u* suffix and an optional *filename* [such as EN{ter}, FE{tch}, KI{ll} and WR{ite}] fail to properly support the *filename* when both options are used simultaneously. [Note: The problem also exists if the *filename:u* form is used; however, the second *:u* suffix is unrelated to the problem.]

In most instances, the optional *:u* suffix is rarely used. When the optional *filename* is not present, the default name from a prior command is used as the file name for the current command:

.FE FOOBAR The explicit file name used sets the default name.

[Some editing operations are performed to update the file contents.]

`.WR` The updated file is written using the default name.

Alternatively, the command

`.WR:1`

writes the file to logical device unit 1 with the same file name.

Since there is no explicit file name in either `WR{ite}` command example, both commands function as expected.

However, if both options are used in the same command such as

`.WR:1 FOOBAR`

the command will fail to complete properly and will likely lead to an error message. [Note: The contents of the edit buffer are not affected by the problem. A similar command with one of the options or no options will succeed.]

P?S/8 Version 8Z Change Log (continued).

The problem was traced to a failure to properly update all internal command parsing data; this can only happen when the simultaneous usage of both options is attempted.

Resolution: As a temporary fix, the proper call to the optional unit scan routine [NEWCAT] is patched in using available locations on the same code page. Future releases will reorganize the code to prevent the problem using less total words.

Implementation: Using P?S/8 BLKODT, the problem is readily fixed:

.BLKODT

0:0007.166 /5401 5376
0:0007.176 /2223 4134
0:0007.177 /7240 5401

S

.

AA6

-

09-May-2017 SIMH Bug Patch.

P?S/8 is generally compatible with the SIMH simulator with the exception of the rewind and unload routines. Until such a time the SIMH problem is solved, apply the following patch:

.BLKODT

0:0006.145 /7650 7600

S

.

(end of file)